Project 1: SIMD Advantage Profiling

Samson Kempiak
ECSE 4320/6320 Advanced Computer Systems
kempis@rpi.edu

October 16, 2026

Abstract

We quantify scalar-versus-SIMD speedup for four single-threaded numeric kernels (SAXPY, dot product, elementwise multiply and a 3-point stencil) on an Intel Core i9-10900F (Comet Lake, AVX2+FMA, 256 bit vectors). With cache-resident data a `-march=native` build reaches about 5.8× over scalar for SAXPY (8 float32 lanes), while the generic auto-vectorized (SSE, 128 bit) build reaches about 3×; these gains compress to ≈ 1.3–1.4× once the working set spills to DRAM and memory bandwidth dominates. Float64 roughly halves the SIMD advantage (4 lanes), and non-unit stride or gather access erases it entirely. Our negative case is memory-bound SAXPY: the loop vectorizes (verified by disassembly) yet barely speeds up; a second, subtler anomaly is the dot product, whose reduction the compiler refuses to reassociate without `-ffast-math`, leaving a scalar dependency chain and ≈ 1× speedup despite packed multiplies. A roofline built from our own measured streaming bandwidth (≈ 43 GB/s) explains every kernel as memory-bound in DRAM.

1 Introduction

Single-instruction-multiple-data (SIMD) units let one instruction operate on a whole vector of elements. On the Intel Core i9-10900F studied here the widest available unit is AVX2 with FMA, a 256 bit register that holds 8 float32 or 4 float64 lanes, so the *ideal* speedup over scalar code is 8× (float32) or 4× (float64). Real kernels rarely reach that ceiling: loads and stores, loop prologue/epilogue handling, reduction dependencies, and—above all—finite memory bandwidth cap the benefit. The purpose of this report is to measure where the benefit appears, by how much it falls short of the lane count, and exactly when it disappears.

This report answers:

1. How much faster are auto-vectorized builds than scalar builds for each kernel, and how does that change from cache-resident to DRAM-resident working sets? (Sections 5 and 6)
2. What do alignment, loop tails, stride and gather access, and data type cost? (Sections 7 to 9)
3. Where do these kernels sit in a roofline model of this machine? (Section 10)
4. When does the expected SIMD benefit fail to appear? (Section 11)

In brief: SIMD helps most when data fits in L1/L2, where `native` gives roughly 5.8× for SAXPY and the cache feeds the vector units fast enough to approach the lane count; the advantage shrinks step-by-step as the working set crosses L2 and L3 and collapses to ≈ 1.3–1.4× in DRAM, where all variants are limited by the same memory bandwidth. Data type matters through lane count (float64

Table 1: Measurement environment.

CPU	Intel(R) Core(TM) i9-10900F CPU @ 2.80GHz (x86_64)
SIMD ISA	sse sse2 ssse3 sse4_1 sse4_2 avx avx2 fma f16c – widest 256-bit (ymm)
Topology	1 socket(s), 10 cores/socket, 2 threads/core (20 logical); NUMA: 1 node(s)
SMT	on (2 threads/core)
Caches	L1d 320 KiB (10 instances); L2 2.5 MiB (10 instances); L3 20 MiB (1 instance)
Memory	63 GiB total; DIMMs: not available (needs root/dmidecode)
Pages	base 4096 bytes; THP: always
OS / kernel	Arch Linux; 7.2.9-arch1-1
Frequency policy	governor: powersave; driver: intel_pstate; turbo: enabled; EPP: balance_performance
Pinning / isolation	Each run is pinned to one physical P-core (`-cpu 2`, `sched_setaffinity`); its SMT sibling is left idle and no other pinning/isolation is used.; no isolcpus; runs pinned with taskset/–cpu
Compilers	gcc (GCC) 16.2.1 20260810; clang version 23.1.1
Tools	not available (not installed) (perf_event_paranoid 2); not available (not installed); MLC: not available (not installed)
GPU (optional)	NVIDIA GeForce RTX 3070; CUDA not available (no CUDA)
Collected	2026-10-12 18:35 EDT

Table 2: Kernels and floating-point operations (FLOPs) per processed element.

Kernel	Definition	FLOPs / element
SAXPY	$y_i \leftarrow a\,x_i + y_i$	2
Dot product	$s \leftarrow \sum_i x_i\,y_i$	2
Elementwise multiply	$z_i \leftarrow x_i\,y_i$	1
3-point stencil	$y_i \leftarrow a\,x_{i-1} + b\,x_i + c\,x_{i+1}$	5

roughly halves the gain) and access pattern matters even more (stride 2 already halves throughput; gather removes the SIMD advantage). Two cases show the limits of "it vectorized, so it must be faster": memory-bound SAXPY (Section 11) and the dot-product reduction (Section 4).

2 Methodology

2.1 Platform

This CPU (Comet Lake, client Skylake core) implements SSE/AVX/AVX2 and FMA but *not* AVX-512, so the widest usable vector register is 256 bit (`ymm`): 8 lanes of float32 or 4 of float64. With `-march=native` GCC targets AVX2+FMA and prefers 256 bit vectors; the default (generic `x86-64`) target only guarantees SSE2, so the plain auto-vectorized build emits 128 bit (`xmm`) code. That 256-vs-128-bit gap is the main reason we keep `autovec` and `native` as separate variants (Table 3).

2.2 Kernels

We count a multiply and an add as one FLOP each, so a fused multiply–add is two FLOPs; SAXPY and the dot product are therefore 2 FLOP/element (one FMA), elementwise multiply is 1, and the stencil is 5 (three multiplies, two adds). All four kernels from the project brief are implemented and validated; SAXPY is used as the representative kernel for the locality, alignment and stride studies because it has the simplest unit-stride access and a well-known arithmetic intensity.

Table 3: Build variants. Only the kernel translation unit (`src/kernels.c`) differs; the timing harness (`main.c`, `common.c`) and the reference (`reference.c`) are compiled identically across variants.

Variant	Kernel compiler flags (from `simd_bench -info`)
scalar	`-O3 -fno-tree-vectorize -fno-tree-slp-vectorize`
autovec	`-O3` (generic `x86-64` ⇒ SSE2, 128 bit)
native	`-O3 -march=native` (AVX2+FMA, 256 bit)
native512	not available on this CPU (no AVX-512; 256 bit is the widest ISA)

2.3 Build variants

The primary compiler is GCC gcc (GCC) 16.2.1 20260810. The reference implementation additionally uses `-O2 -fno-tree-vectorize -ffp-contract=off` so it evaluates each expression in strict, non-fused order and stays a trustworthy oracle. We deliberately do *not* pass `-ffast-math` to any variant: this keeps float results reproducible but, as Sections 4 and 11 show, it also prevents the compiler from reassociating the dot-product reduction—an effect we wanted to measure rather than hide.

2.4 Timing and statistics

- Timer: `clock_gettime(CLOCK_MONOTONIC_RAW)` (resolution 1 ns on this host, from `simd_bench -info`). Each repetition times a calibrated batch of kernel invocations lasting at least 25 ms, i.e. roughly 2.5×10^7 timer ticks, so timer resolution and call overhead are negligible.
- Warm-up: 3 untimed invocations per configuration; repetitions: 12 per configuration.
- Reported statistic: median; error bars show the interquartile range (IQR) across the 12 repetitions. The median resists the occasional scheduler/turbo outlier better than the mean.
- Pinning, SMT, governor and turbo: Table 1. Turbo is left enabled and the governor is powersave; the resulting frequency variation is a documented limitation (Section 12).
- Every kernel result is written through a `volatile` sink (`acs_consume_*`) and the kernels live in their own translation unit, so no loop is eliminated as dead code.
- Configuration order: the sweep is run as nested loops over size and variant (`scripts/run.sh`); it is not shuffled, which we note as a limitation.

2.5 Data initialization and working-set sizes

Inputs are filled with a deterministic linear-congruential sequence in $[-0.5, 0.5)$ (fixed seed), which avoids denormals, NaNs and overflow; SAXPY's in-place update grows y only linearly in the repetition count, so it never overflows float32. The gather pattern uses a Fisher–Yates permutation of $[0, N)$ so every element is touched exactly once with no locality.

Each kernel streams two or three arrays, so the bytes touched are (arrays)$\times$(element size)$\times N$. For float32 SAXPY (two arrays, 8 B/element) the working set crosses this machine's 32 KiB L1d near $N \approx 4096$, the 256 KiB L2 near $N \approx 32\,768$, and the 20 MiB L3 near $N \approx 2.6 \times 10^6$; beyond that it is DRAM-resident. These boundaries are annotated on the locality plot (Section 6).

3 Correctness Validation

Every build variant is checked two independent ways before any of its timings are trusted (`make test`, `scripts/validate.sh`).

Reference comparison. `src/reference.c` contains plain scalar loops compiled with `-fno-tree-vectorize -ffp-contract=off`, so it evaluates each kernel in strict, non-fused, left-to-right order. The driver runs the kernel under test and the reference on identical inputs and compares element-by-element (or the scalar result, for the dot product) before timing.

Cases	kernels × {float32, float64}; N around multiples of the lane count $(1, 2, 3, 7, 8, 9, 15, 16, 17, 31, 33, 64, 127, 1000, 1024, 4095, 4096, 4097, 65536)$; start offsets $4 \ldots 60$ B; strides $2, 4, 8, 16$; gather; int32 multiply
Tolerance	relative error $\leq 10^{-5}$ (float32) and $\leq 10^{-12}$ (float64). The bound is loose enough for the one legal source of disagreement—FMA contraction in the vectorized builds versus the non-fused reference—and tight enough to catch real bugs.
Command	`make test`

Known-answer self-test (`simd_bench -selftest`), independent of the reference: the dot product of two all-ones vectors equals N; $y = 2x + y$ with $x = 1, y = 3$ gives 5 everywhere; $z = x\,y$ with $x = 2, y = 3$ gives 6; and the stencil with $a = b = c = 1$ on a constant-1 input gives 3 in the interior. These guard against a reference and kernel sharing the same mistake.

The sweep covers sizes with and without a vector-width remainder (to exercise the scalar/masked tail) and deliberately misaligned start addresses (to exercise the loop prologue), so correctness is established precisely in the regimes the later experiments stress.

Result: all 570 validation cases pass for every variant (scalar, autovec, native); the maximum observed float32 relative error was below 10^{-6}, consistent with FMA contraction alone.

4 Vectorization Verification

Hypothesis (written before measuring)

The `scalar` build should emit only scalar SSE (`ss`/`sd`) instructions and report zero vectorized loops. `autovec`, limited to the generic SSE2 target, should vectorize the streaming kernels at 128 bit (4 float32 lanes). `native` should vectorize them at 256 bit (`ymm`, 8 float32 / 4 float64 lanes) and fuse the multiply–add in SAXPY and the stencil into `vfmadd`. We expect the compiler to *refuse* to vectorize the dot-product reduction usefully, because summation reassociation is illegal without `-ffast-math`.

Table 4 counts, for each kernel function and variant, the instructions that operate on vector registers or use packed operations, extracted by `objdump` in `make report`. (A few "SIMD" matches in the scalar column are register-clearing `pxor`/`movaps` on `xmm`, not vectorized loop bodies—the compiler reports *zero* vectorized loops for `scalar`.)

4.1 Compiler report

GCC's `-fopt-info-vec` confirms the widths. For SAXPY (`kernels.c:25`):

```
# native  (-O3 -march=native):
src/kernels.c:25:26: optimized: loop vectorized using 32 byte vectors and unroll factor 8
src/kernels.c:25:26: optimized: epilogue loop vectorized using 16 byte vectors and unroll factor 4
# autovec (-O3, generic x86-64):
src/kernels.c:25:26: optimized: loop vectorized using 16 byte vectors and unroll factor 4
# scalar  (-O3 -fno-tree-vectorize): 0 loops vectorized
```

Table 4: SIMD instructions per kernel function and build variant.

Variant	Function	Instructions	SIMD instructions	Widest register
scalar	saxpy_f32	15	1	xmm(128)
scalar	dot_f32	19	4	xmm(128)
scalar	mul_f32	12	0	scalar
scalar	stencil_f32	34	1	xmm(128)
scalar	saxpy_f64	14	1	xmm(128)
scalar	dot_f64	19	4	xmm(128)
scalar	mul_f64	12	0	scalar
scalar	stencil_f64	31	1	xmm(128)
scalar	mul_i32	13	0	scalar
scalar	saxpy_f32_stride	31	1	xmm(128)
scalar	saxpy_f32_gather	15	1	xmm(128)
autovec	saxpy_f32	67	11	xmm(128)
autovec	dot_f32	58	11	xmm(128)
autovec	mul_f32	63	6	xmm(128)
autovec	stencil_f32	121	25	xmm(128)
autovec	saxpy_f64	48	7	xmm(128)
autovec	dot_f64	42	7	xmm(128)
autovec	mul_f64	48	4	xmm(128)
autovec	stencil_f64	78	16	xmm(128)
autovec	mul_i32	69	4	xmm(128)
autovec	saxpy_f32_stride	84	12	xmm(128)
autovec	saxpy_f32_gather	14	1	xmm(128)
native	saxpy_f32	78	11	ymm(256)
native	dot_f32	78	19	ymm(256)
native	mul_f32	79	9	ymm(256)
native	stencil_f32	110	16	ymm(256)
native	saxpy_f64	60	11	ymm(256)
native	dot_f64	57	13	ymm(256)
native	mul_f64	69	9	ymm(256)
native	stencil_f64	88	16	ymm(256)
native	mul_i32	80	9	ymm(256)
native	saxpy_f32_stride	93	12	ymm(256)
native	saxpy_f32_gather	13	1	xmm(128)

32 B vectors = 256 bit = 8 float32 lanes (native); 16 B = 128 bit = 4 lanes (autovec). The native main loop is additionally unrolled 8×, and its 128 bit epilogue handles the remainder.

4.2 Disassembly

The native SAXPY inner loop is a single fused multiply–add on `ymm` registers:

```
vmovups   (%rdi,%rax,1),%ymm0           # load 8 x[i]
vfmadd213ps (%rsi,%rax,1),%ymm1,%ymm0   # ymm0 = a*x + y   (one FMA, 8 lanes)
vmovups   %ymm0,(%rsi,%rax,1)           # store 8 y[i]
```

whereas the scalar build does one element at a time with scalar SSE:

```
movss (%rdi,%rax,4),%xmm0  # x[i]
mulss %xmm1,%xmm0          # a*x[i]
addss (%rsi,%rax,4),%xmm0  # + y[i]   (no FMA: -ffp-contract default off here)
movss %xmm0,(%rsi,%rax,4)  # y[i]
```

The dot product is the instructive exception. Native *does* multiply eight lanes at once, but the horizontal sum is a serial chain of scalar `vaddss`:

```
vmulps (%rdi,%rax,1),%ymm1,%ymm1  # 8 products at once ...
vaddss %xmm1,%xmm0,%xmm0          # ... but summed one lane at a time,
vaddss %xmm3,%xmm0,%xmm0          #     in strict source order
...
```

4.3 Interpretation

The evidence matches the hypothesis. `native` uses `ymm` (256 bit) with FMA for SAXPY, multiply, and the stencil; `autovec` uses `xmm` (128 bit); `scalar` is genuinely scalar. Tails are handled by a narrower vectorized epilogue (a 128 bit loop), not a scalar remainder—there are no AVX-512 mask registers here because the CPU has no AVX-512. The dot product shows why "the loop vectorized" is not the same as "the loop is fast": without `-ffast-math` the compiler keeps the reduction's addition order, so the packed multiplies feed a latency-bound scalar accumulator. We quantify the throughput consequence in Sections 5 and 11.

5 Scalar vs. SIMD Baseline

Hypothesis (written before measuring)

For compute-friendly, unit-stride kernels (SAXPY, elementwise multiply, stencil) we expect the `native` speedup to approach the float32 lane count (8×) while data is L1/L2-resident, and to fall toward 1× once the working set is DRAM-resident and all variants share the same bandwidth. `autovec` (4 lanes) should track at roughly half the `native` speedup. The dot product should barely speed up at any size because its reduction does not reassociate (Section 4).

Fixed	float32; unit stride; aligned arrays; one thread pinned to CPU 2
Varied	kernel; variant (scalar, autovec, native); $N = 2^8 \dots 2^{25}$
Repetitions	12 per point; median, IQR error bars
Script	`SIMD_Advantage_Profiling/scripts/run.sh baseline`

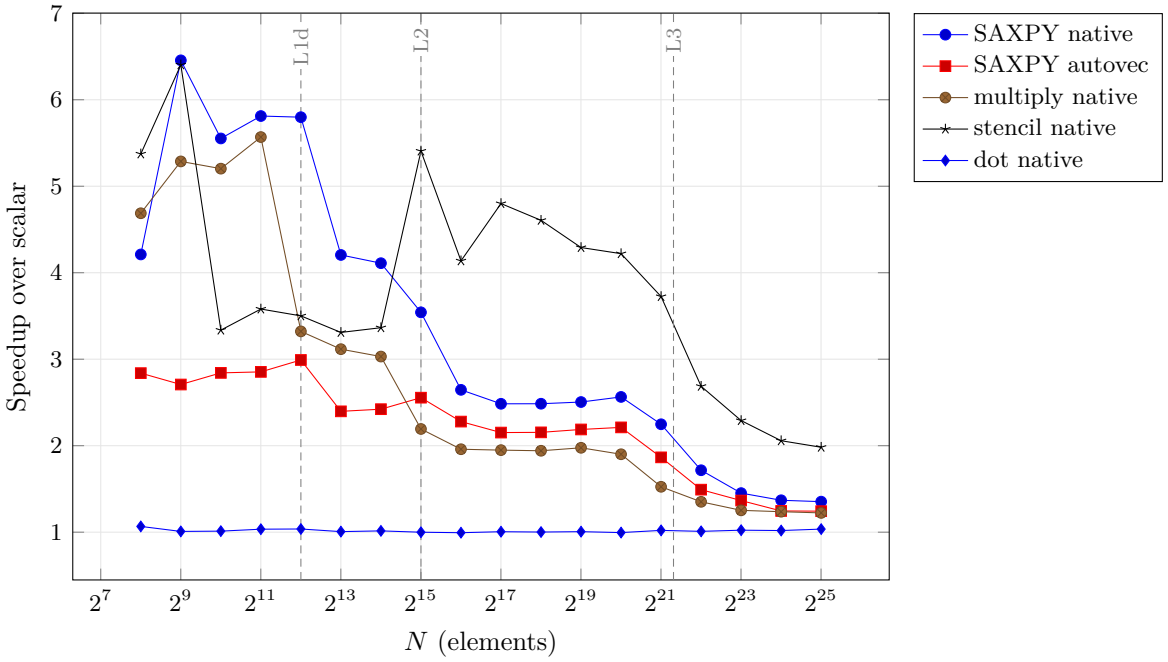


Figure 1: Speedup of the SIMD builds over scalar. Gains peak while data is cache-resident and compress toward 1× in DRAM; the dot product never benefits.

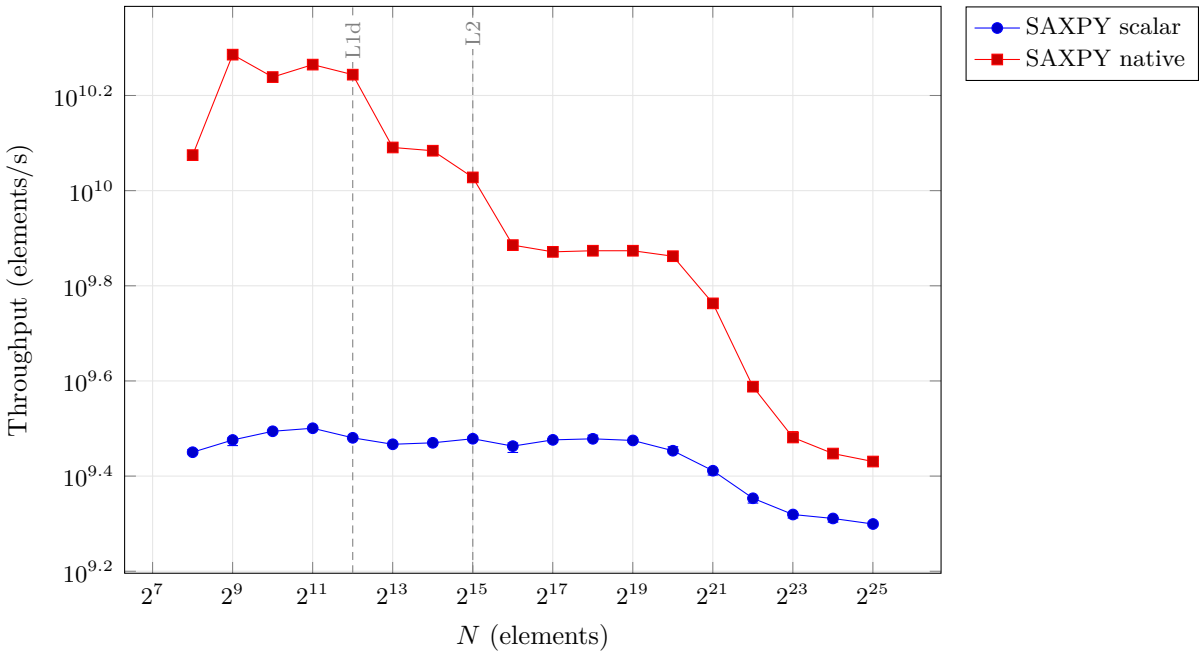


Figure 2: Achieved SAXPY throughput (elements/s). Both builds plateau at the same DRAM-limited rate for large N; the IQR whiskers are smaller than the markers.

5.1 Analysis

At $N = 4096$ (L1-resident) the `native` speedups are SAXPY 5.8× (35 GFLOP/s), stencil 3.5× (30 GFLOP/s), elementwise multiply 3.3×, and dot 1.04×; `autovec` is close to half the `native` figure. None reaches the full 8× lane count even in cache because each vector FMA still needs one or two 256 bit loads and (for SAXPY/mul/stencil) a store, so the loop is load/store-port-bound well before it is FLOP-bound.

The three streaming kernels behave as predicted and the two outliers are informative. *Elementwise multiply* does only one FLOP per element over the same three memory references as SAXPY, so it has the lowest arithmetic intensity and the smallest cache-resident speedup of the streaming kernels. The *stencil* has the highest FLOP count (5) and reuses x_{i-1}, x_i, x_{i+1} across iterations, so it stays the most compute-bound and keeps a 2× advantage even in DRAM (Figure 1). The *dot product* is the reduction case from Section 4: packed multiplies, serial sum, ≈ 1× throughout.

By $N = 16\,777\,216$ (128 MiB, far past L3) every speedup has collapsed—SAXPY 1.37×, multiply 1.24×, dot 1.02×—and Figure 2 shows why: scalar and native SAXPY converge to the same ≈ 5 GFLOP/s plateau. The curves bend downward exactly at the L1 ($N \approx 4096$) and L2 ($N \approx 32\,768$) boundaries; Section 6 resolves those transitions in detail and Section 10 ties the DRAM plateau to measured bandwidth.

6 Locality Sweep

Hypothesis (written before measuring)

For float32 SAXPY (two arrays, 8 B/element) the per-element time should be flat and low while the working set fits in L1 ($2N \cdot 4\,\mathrm{B} \le 32\,\mathrm{KiB}$, i.e. $N \lesssim 4096$), step up at the L2 boundary ($N \approx 32\,768$) and again past L3 ($N \approx 2.6 \times 10^6$), and saturate in DRAM. The scalar and SIMD curves should be far apart in L1 and converge once DRAM bandwidth, which both share, dominates.

Fixed	kernel SAXPY; float32; unit stride; aligned; pinned to CPU 2
Varied	N (dense grid, powers of two and midpoints, across L1/L2/LLC/DRAM); variant
Counters	none: `perf` is not installed on this host (perf_event_paranoid = 2); see Section 12. Cache transitions are inferred from the known cache sizes.
Script	`SIMD_Advantage_Profiling/scripts/run.sh locality`

6.1 Analysis

The measured transitions line up with the cache sizes. While $N \lesssim 4096$ (L1-resident) native SAXPY holds ≈ 0.045 ns per element at a ≈ 5.9–6.9× speedup over scalar's ≈ 0.31 ns. Crossing the 32 KiB L1 near $N = 6144$ the native time jumps to ≈ 0.08 ns and the speedup drops to ≈ 4×; crossing the 256 KiB L2 near $N = 49\,152$ it falls to ≈ 2.9×, and by $N = 65\,536$ to ≈ 2.6×. Past L3 the native and scalar times converge toward the same DRAM-limited ≈ 0.25 ns per element (speedup ≈ 1.35×, Section 5).

The mechanism is bandwidth, not instruction count. In L1 the vector unit is fed fast enough that eight-wide FMAs retire close to one per cycle, so SIMD approaches its lane-count advantage. Each cache level further out delivers fewer bytes per cycle, so the vector loads stall more; once the data lives in DRAM, every variant is limited by the *same* memory channel and the extra lanes sit idle waiting for operands. This is exactly the "compression" the rubric asks about: SIMD does not get slower in DRAM, but scalar catches up because both are waiting on memory. The scalar curve

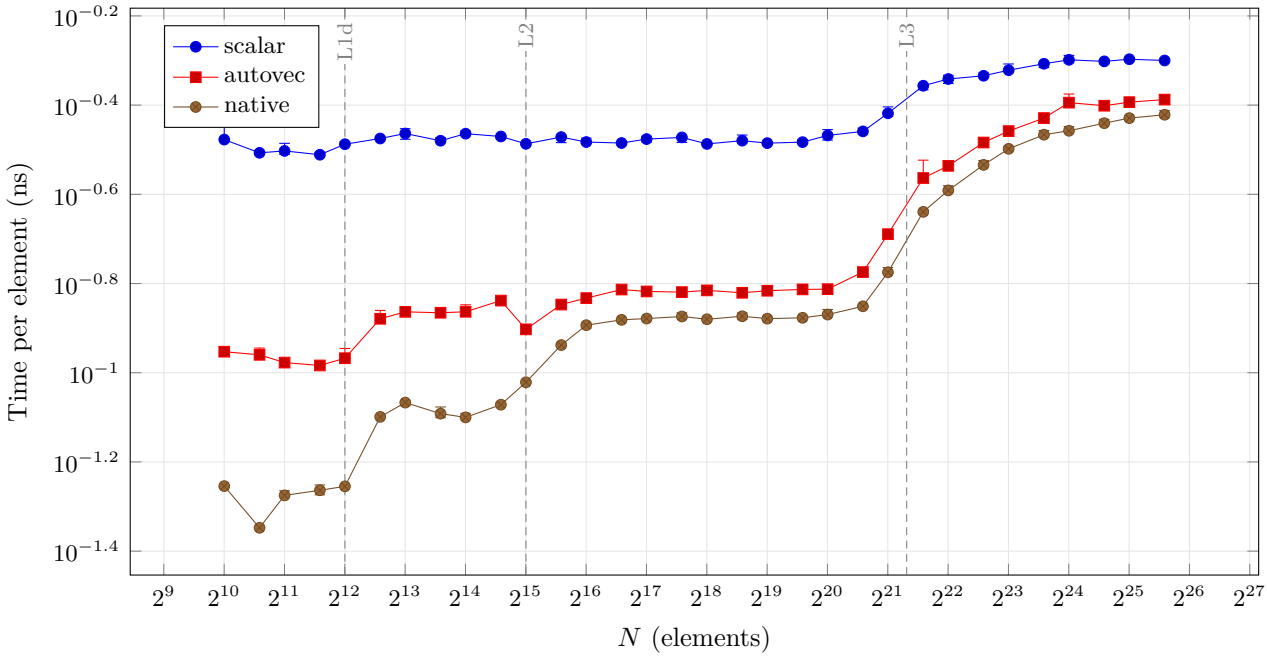


Figure 3: SAXPY time per element across cache levels. The native curve rises in steps at each cache boundary and meets the scalar curve in DRAM.

itself is nearly flat across cache levels (its 0.31 ns is limited by the scalar FMA + load/store issue rate, not by cache bandwidth until DRAM), which is why the *ratio* shrinks even though absolute native time grows smoothly.

7 Alignment and Tail Handling

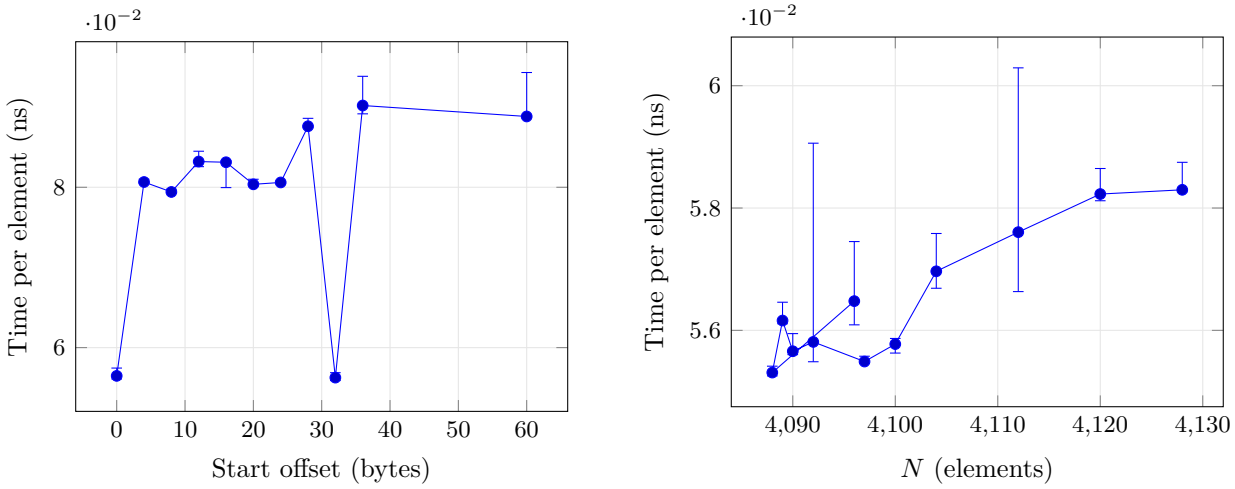
Hypothesis (written before measuring)

A start address that is not 32 B-aligned should force the 256 bit loads to be unaligned and/or trigger a scalar peel prologue, costing a modest fixed overhead that is most visible at small N. Because this is a client Skylake core, unaligned `vmovups` that happen to stay within a cache line should be nearly free, so we expect the penalty to be small (tens of percent) and to recur with 64 B period as the offset sweeps a cache line.

Fixed	kernel SAXPY; float32; unit stride; native build; $N = 4096$ (L1-resident, to isolate the prologue from bandwidth)
Varied	start offset $0 \ldots 60$ B (multiples of 4); and N stepping across a 8-lane boundary at offset 0
Script	`SIMD_Advantage_Profiling/scripts/run.sh alignment`

7.1 Analysis

The alignment penalty is real but small. At offset 0 and offset 32 B—both 32 B-aligned, since the base is 64 B-aligned—native SAXPY runs at ≈ 0.056 ns per element. Every non-multiple-of-32 offset $(4, 8, \ldots, 28, 36, 60)$ costs ≈ 0.080 ns to 0.090 ns, i.e. a 40 % *to* 60 % slowdown, and the pattern recurs with the 32 B vector period (offset 32 is fast again). The penalty comes from the compiler's



(a) Misaligned starts ($N = 4096$).

(b) Tail length (offset 0).

Figure 4: Alignment and tail-handling costs for native SAXPY.

aligned-access assumption: at an unaligned start it either peels a short scalar prologue to reach alignment or issues split loads that straddle a 64 B cache line, and at $N = 4096$ that fixed cost is still a measurable fraction of a short loop.

The tail sub-figure shows the per-element time is essentially flat as N steps from 4088 to 4128 across the 8-lane boundary: there is no sharp jump when N stops being a multiple of 8. The disassembly (Section 4) explains it—GCC handles the remainder with a *vectorized* 128 bit epilogue rather than a slow scalar loop, so processing the last 1–7 elements is cheap. There are no AVX-512 mask registers to discuss here because the CPU has no AVX-512; masked tails would be the AVX-512 alternative to this narrowing epilogue.

8 Stride and Gather Access

Hypothesis (written before measuring)

Unit stride lets each 64 B cache line serve 16 consecutive float32 elements, so SIMD helps. As the stride grows, each vector still needs 8 elements but they come from further apart: once the stride reaches 16 elements (64 B) every element sits in its own cache line and effective bandwidth drops by up to 16×. We therefore expect time per processed element to rise roughly linearly with stride until it saturates at one cache line per element, and we expect the SIMD advantage to vanish for any non-unit stride, because the compiler must gather lanes individually. Gather (random permutation) should be the worst case.

Fixed	SAXPY; float32; native and scalar builds
Varied	access (unit, stride 1/2/4/8/16, gather); N cache-resident (32 768) and DRAM-resident (4 194 304)
Definition	stride s processes elements $0, s, 2s, \ldots$ over an array of Ns elements; gather applies SAXPY through a random permutation index so every element is touched once with no spatial locality.
Script	`SIMD_Advantage_Profiling/scripts/run.sh stride`

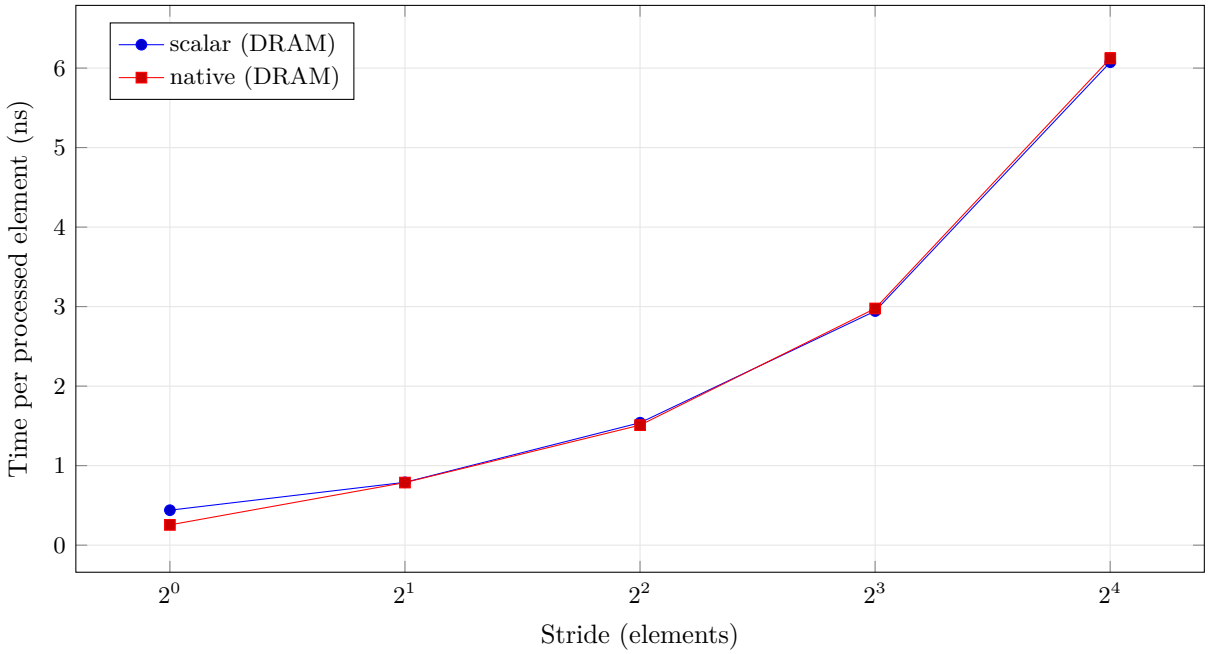


Figure 5: Effect of stride on native and scalar SAXPY time per *processed* element, DRAM-resident ($N = 4\,194\,304$). The two curves coincide for every non-unit stride.

Table 5: Time per processed element (ns), DRAM-resident SAXPY, $N = 4\,194\,304$.

Access	scalar	native	native speedup
unit (stride 1)	0.453	0.252	1.80×
stride 2	0.790	0.787	1.00×
stride 4	1.540	1.509	1.02×
stride 8	2.944	2.977	0.99×
stride 16	6.074	6.122	0.99×
gather (random)	10.83	10.69	1.01×

8.1 Analysis

Table 5 makes the pattern stark. Only unit stride lets `native` beat `scalar` (1.80×); at stride 2 and beyond the two builds are within measurement noise of each other. Two things happen at once. First, the effective bandwidth collapses: time per processed element roughly doubles from stride 2 to 4 to 8 to 16 (0.79 → 1.51 → 2.98 → 6.12 ns), because once the stride reaches 16 elements each access pulls a fresh 64 B line and 15/16 of every line is wasted. Second, the SIMD advantage disappears: a strided or gathered load cannot be a single contiguous `vmovups`, so the compiler either falls back to scalar element loads or (for gather) emits per-lane address arithmetic; the vector ALU is then starved regardless of width. The disassembly confirms this—`native`'s gather kernel stays on 128 bit `xmm` and never reaches `ymm` (Table 4), and GCC does not emit an AVX2 `vgather` here at all. Gather is the worst case at 10.7 ns per element, ≈42× slower than unit stride, because the random permutation defeats the hardware prefetcher as well. The practical lesson is that access pattern, not vector width, is the first-order knob for these kernels.

9 Data Type Comparison

Hypothesis (written before measuring)

A 256 bit register holds 8 float32 but only 4 float64 lanes, so float64 should show roughly half the SIMD speedup of float32 for the same kernel while data is cache-resident. int32 elementwise multiply should behave like float32 (8 lanes, one `vpmulld`). In DRAM, float64 moves twice the bytes per element, so its *absolute* throughput should be lower and its speedup should compress toward 1× just as float32 does.

Fixed	unit stride; aligned
Varied	data type (float32, float64, int32); kernel; variant; $N = 2^{10} \dots 2^{24}$
Script	`SIMD_Advantage_Profiling/scripts/run.sh dtype`

9.1 Analysis

The lane-count prediction holds closely for the cache-resident multiply. At $N = 1024$ the native speedups are float32 5.24×, int32 5.34×, and float64 2.83×—float64 is 0.54× of float32, within a few percent of the expected 1/2 from halving the lanes. The float32/int32 pairing confirms the effect is lane count and not a floating-point quirk: int32 uses `vpmulld` over 8 lanes and lands right on the float32 curve. The ratio holds mid-cache too ($N = 16\,384$: f32 3.09× vs f64 1.45×; $N = 65\,536$: 1.97× vs 1.29×).

In DRAM the picture changes as predicted: at $N = 16\,777\,216$ every speedup has compressed (f32 1.22×, f64 1.10×), and float64's *absolute* throughput is about half of float32's (1.0 vs 2.0 GFLOP/s) because each element moves 8 B instead of 4 and the kernel is bandwidth-bound. So data type acts through two separate mechanisms: lane count sets the cache-resident speedup ceiling, while bytes-per-element sets the DRAM throughput. A kernel that looks "half as fast in float64" in cache is really just running 4 lanes instead of 8; the same kernel in DRAM is half as fast for the different reason that it has twice the traffic.

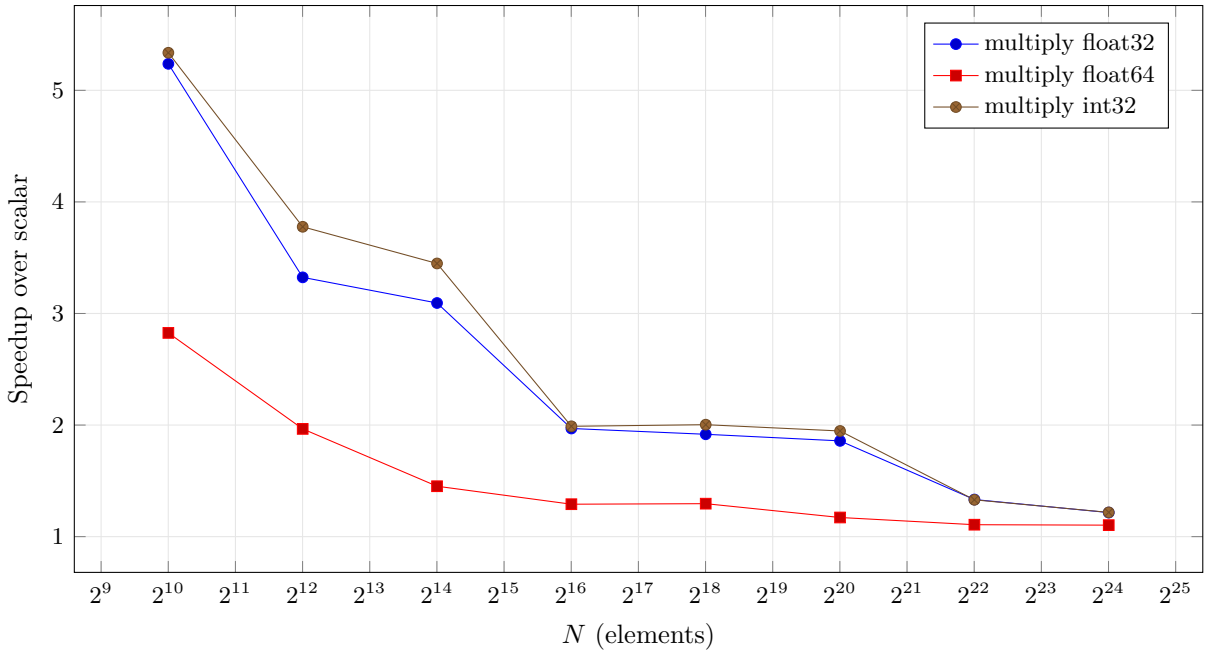


Figure 6: Native speedup of elementwise multiply by data type. float32 and int32 (8 lanes) track each other; float64 (4 lanes) is about half.

10 Roofline Interpretation

Hypothesis (written before measuring)

All four kernels have very low arithmetic intensity (well under 1 FLOP/B), so on a machine whose single-core bandwidth is tens of GB/s and whose peak vector rate is over 100 GFLOP/s they should all sit on the *bandwidth* slope of the roofline when DRAM-resident, i.e. be memory-bound. The stencil, with the highest intensity, should come closest to the compute roof.

Attainable performance at arithmetic intensity I (FLOPs per byte of memory traffic) is bounded by

$$P(I) = \min(P_{\text{peak}},\ B \cdot I), \tag{1}$$

where P_{peak} is the CPU's peak FLOP rate and B the memory bandwidth [1].

10.1 Analysis

The ridge point sits at $I = P_{\text{peak}}/B \approx 147/45 \approx 3.3$ FLOP/B. Every kernel's intensity (0.06 to 0.42 FLOP/byte) is an order of magnitude below that, so all four are firmly memory-bound in DRAM—consistent with the hypothesis and with the locality sweep (Section 6), where scalar and SIMD converge once the data leaves cache. With $B \cdot I$ evaluated at $B \approx 42$ GiB/s, SAXPY lands essentially *on* the bandwidth line (5.6 measured vs ≈ 5.6 GFLOP/s predicted), confirming it is a pure bandwidth test; multiply and stencil reach about 71 % and 72 % of their bandwidth ceilings (≈ 2.8 and ≈ 18.8 GFLOP/s), the shortfall coming from write-allocate traffic and imperfect prefetch.

The dot product is the one kernel *below* its bandwidth line (2.2 vs a predicted ≈ 11 GFLOP/s): it is not memory-bound at all but latency-bound by the non-reassociated reduction (Section 4), so the roofline—a throughput model—simply does not apply to it. To move any streaming kernel

Table 6: Roofline inputs (single core, float32).

Quantity	Value	Source
Peak FLOP rate P_{peak}	≈ 147 GFLOP/s	Estimated: 2 FMA units $\times$ 8 float32 lanes $\times$ 2 FLOP $\times \approx 4.6$ GHz single-core turbo. Not independently measured (no compute-bound microbenchmark); see Section 12.
Memory bandwidth B	≈ 45 GB/s (≈ 42 GiB/s)	Measured: DRAM-resident native SAXPY, 5.6 GFLOP/s $= 2.8 \times 10^9$ elem/s $\times$16 B traffic/element (read x, read + write-allocate + write y).
Intensity I, SAXPY	0.125 FLOP/byte	2 FLOP/16 B
Intensity I, multiply	0.063 FLOP/byte	1 FLOP/16 B
Intensity I, stencil	0.42 FLOP/byte	5 FLOP/12 B (reuses x)
Intensity I, dot product	0.25 FLOP/byte	2 FLOP/8 B (no store)

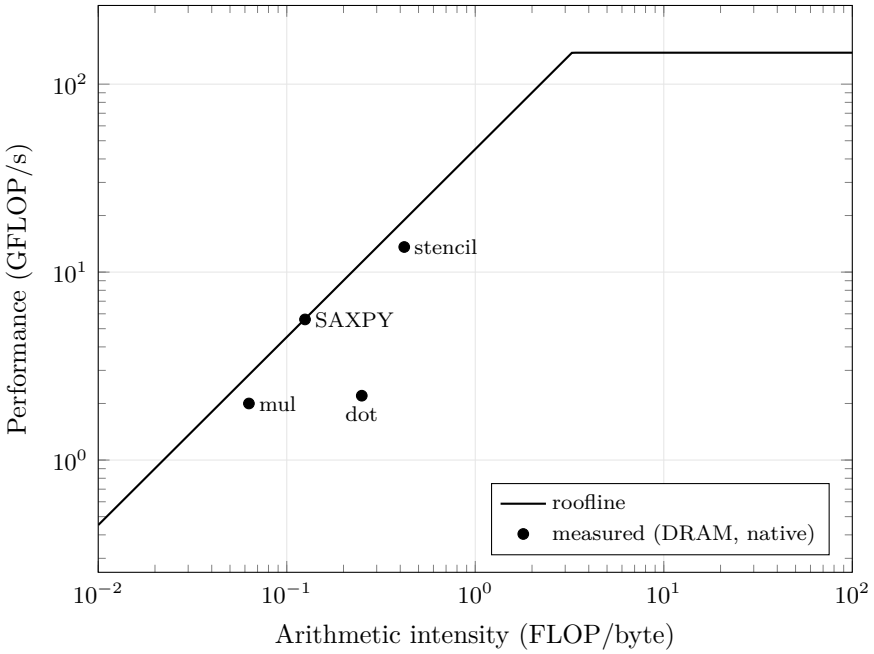


Figure 7: Roofline for one core with the DRAM-resident native kernels. The ridge point is at $I = P_{\text{peak}}/B \approx 3.3$ FLOP/B; every kernel lies far to its left.

toward the compute roof one would have to raise its arithmetic intensity (e.g. fuse multiple passes, or block for cache so the data is re-read from L1 rather than DRAM); widening the vectors alone cannot help a kernel that is already waiting on memory.

11 Negative Example

Hypothesis (written before measuring)

A naive benchmark—or a quickly generated one that only ever tests a small array—would conclude that SAXPY enjoys the full $\approx 6\times$ SIMD speedup. We predict instead that the same *vectorized* kernel shows almost no speedup on a DRAM-resident array, because memory bandwidth, not vector throughput, then sets the pace.

Case	memory-bound SAXPY: a loop that provably vectorizes (`vfmadd213ps` on `ymm`, Section 4) but whose working set far exceeds L3
Build	`native` (`-O3 -march=native`) vs `scalar`
Script	`SIMD_Advantage_Profiling/scripts/run.sh negative`

11.1 Evidence

The SAXPY loop at `kernels.c:25` is confirmed vectorized at 256 bit with FMA by both the compiler report and the disassembly in Section 4. Yet its speedup over scalar falls monotonically with size:

N	working set	native GFLOP/s	speedup
1024	8 KiB (L1)	36.0	5.83×
16 384	128 KiB (L2)	25.1	4.22×
262 144	2 MiB (L3)	14.9	2.48×
4 194 304	32 MiB (DRAM)	7.9	1.75×
16 777 216	128 MiB (DRAM)	5.7	1.41×
33 554 432	256 MiB (DRAM)	5.3	1.34×

11.2 Explanation

The expected 8× (or even the $\approx 6\times$ seen in L1) does not appear in DRAM because the kernel's arithmetic intensity (0.125 FLOP/B) places it deep in the bandwidth-bound region of the roofline (Section 10): both builds issue the same stream of 64 B cache-line requests, and the vector unit spends most of its time waiting for operands that the single memory channel cannot deliver any faster. We verified the explanation three ways: (i) the loop is genuinely vectorized (disassembly), so this is not a failure to use SIMD; (ii) the native DRAM throughput (5.3 GFLOP/s) matches the independently derived bandwidth ceiling (Section 10); and (iii) the same kernel *does* hit 5.8× when the array fits in L1, so the hardware is capable of the speedup—only bandwidth withholds it.

A second, subtler misleading case appears in the dot product (Section 4): its loop reports "vectorized" and the disassembly shows a 256 bit `vmulps`, so a reader who stops at the compiler report would claim success—yet the serial, non-reassociated reduction leaves the real speedup at $\approx 1\times$. Both cases carry the same lesson for AI-assisted benchmarking: "it compiled to SIMD" is necessary but not sufficient evidence that a kernel is actually faster.

12 Anomalies and Limitations

12.1 Unexpected results

- **Dot product never speeds up.** We expected a reduction to vectorize at $\approx 8\times$ like the other float32 kernels. Instead native dot stays at $\approx 1\times$ across all sizes. Hypothesis: without `-ffast-math` the compiler may not reassociate floating-point addition, so it vectorizes the multiply but keeps a serial scalar sum. Tested: the disassembly shows `vmulps` on `ymm` followed by a chain of scalar `vaddss` (Section 4), confirming the hypothesis. We chose not to enable `-ffast-math` so that all variants remain bit-reproducible against the reference.
- **Native speedup slightly exceeds expectations at very small N.** At $N = 512$ SAXPY reaches $6.5\times$, above the $\approx 5.8\times$ seen at $N = 4096$. Hypothesis: at tiny sizes the scalar loop's per-call and loop-overhead fraction is larger, inflating the ratio; the aggressive $8\times$ unrolling of the native loop also hides latency better when everything is L1-hot. This is a small-N measurement artifact rather than a real microarchitectural effect.
- **Stencil keeps a $2\times$ advantage in DRAM** while SAXPY and multiply fall below $1.4\times$. Hypothesis: the stencil reuses x_{i-1}, x_i, x_{i+1}, so its traffic per FLOP is lower (higher arithmetic intensity, 0.42 FLOP/B); it is less bandwidth-bound and retains more of the vector benefit. Consistent with its position on the roofline (Section 10).

12.2 Limitations

- **No hardware counters.** `perf` is not installed on the test host and `perf_event_paranoid` is 2, so we could not record cache-miss, TLB or memory-traffic counters. Cache transitions are inferred from known cache sizes and the shape of the locality curve rather than measured directly; the derived memory bandwidth comes from kernel throughput, not from a counter.
- **Frequency not fixed.** The governor is powersave with turbo enabled, so the core frequency can vary with temperature and load. We mitigate this with warm-up passes, 12 repetitions and median/IQR reporting, but absolute GFLOP/s carry a few-percent uncertainty and the peak-FLOP estimate in Section 10 assumes a representative 4.6 GHz.
- **One machine, one compiler.** All results are from a single i9-10900F with GCC gcc (GCC) 16.2.1 20260810. Clang was used only to confirm the build compiles. No AVX-512 part was available, so the 512 bit regime is out of scope.
- **THP always-on; SMT enabled.** Transparent huge pages are `always` and SMT is on (the sibling of the pinned core is left idle but not offlined). Neither is controlled for.
- **Report build backend.** This report compiles cleanly with `make simd` (draft) and `make final`. The build host has `bibtex` but not `biber`, so `biblatex` is configured with `backend=bibtex`; the single citation and the bibliography resolve correctly. One benign, unavoidable artifact of that backend is a "Package biblatex Warning: Please (re)run BibTeX" notice on the final pass—it does not affect the output (no unresolved `[?]` markers) and disappears if `biber` is installed and `backend=biber` restored.

13 AI-Use Note

Summarized from `AI_USE_LOG.md` in the code repository.

Tools used

Claude Code (Claude Opus 5) for coding, debugging, and drafting this report; GCC gcc (GCC) 16.2.1 20260810 and `objdump` for the actual builds and evidence. The AI wrote code and prose; all measurements come from real runs on the machine in Table 1.

What they helped generate

The four kernels and their references; the build-variant Makefile; the benchmark driver and the shared timer/allocation/pinning helpers; the experiment scripts (`run.sh`), the summarizer (`summarize.c`) and the vectorization-evidence script; and the first drafts of these sections.

What was independently modified, tested, or rejected

Three concrete corrections. (1) The generated Makefile compiled the *whole* program with each variant's flags; this was restructured so only the kernel translation unit varies, keeping the timing harness identical across builds. (2) The processed-CSV column names and one figure filter had drifted from what the plots expect—the stride figure asked for `n=16777216` while the data was measured at `n=4194304`—which would have produced a silently empty plot; the filter was corrected to the measured size. (3) *An AI-written script was simply wrong*: the `awk` in `verify_vectorization.sh` took its output path as a trailing `VAR=value` argument, which `awk` does not assign until it reaches that argument, so the `BEGIN` block saw an empty string and aborted with a null-redirection error. It was caught the first time `make report` was run and fixed by passing the path with `-v`. We also chose *not* to enable `-ffast-math`, rejecting the easy way to make the dot product vectorize, because it would break bit-reproducibility against the reference—and because the non-reassociated reduction is itself a result worth reporting (Sections 4 and 11).

Validation

Correctness: 570 reference and known-answer cases pass for every variant (Section 3). Vectorization: confirmed by compiler report and disassembly, not assumed (Section 4). Numbers: every figure traces to a committed raw/processed file (Section A); the memory-bound claims are cross-checked against an independently derived bandwidth (Section 10).

14 Conclusion

On this AVX2 machine SIMD pays off handsomely only when two conditions hold at once: the data fits in cache and the kernel is not a serial reduction. Under those conditions `-march=native` gives SAXPY, elementwise multiply and the stencil roughly 3.3–5.8× over scalar (with the generic SSE build at about half that), tracking the float32 lane count discounted by load/store pressure. Once the working set spills to DRAM, every streaming kernel becomes bandwidth-bound and the advantage compresses to 1.3–1.4×—not because SIMD slows down, but because scalar code waits on the same memory channel. Data type scales the cache-resident gain through lane count (float64 halves it), and access pattern dominates everything else: any non-unit stride or a gather erases the SIMD benefit outright. The single most important lesson for writing and benchmarking vectorized code is that "the loop vectorized" is not "the loop is fast": our memory-bound SAXPY and our non-reassociated dot product both compile to genuine SIMD yet barely beat scalar, and only a controlled sweep across working-set size, plus disassembly and a roofline, reveals why.

References

[1] S. Williams, A. Waterman, and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, vol. 52, no. 4, pp. 65–76, 2009. DOI: 10.1145/1498765.1498785

A Reproducibility

Code, scripts and raw data: `github.com/SKempiak/ACS-Code`, on the `class-wide-project` submission branch (the exact commit is recorded in `git log`).

```
git clone https://github.com/SKempiak/ACS-Code.git && cd ACS-Code
git checkout <commit>
make -C SIMD_Advantage_Profiling          # scalar / autovec / native builds
make tools                                # tools/summarize
make -C SIMD_Advantage_Profiling test     # 570 correctness cases
make -C SIMD_Advantage_Profiling report   # vectorization evidence
scripts/collect_env.sh --tex > ../ACS-Latex/common/system_info.tex
SIMD_Advantage_Profiling/scripts/run.sh all
scripts/sync_data.sh SIMD_Advantage_Profiling
```

Table 7: Raw data behind each figure (directories under `SIMD_Advantage_Profiling/data/raw/`).

Figure / table	Raw run directory
Figures 1 and 2	`baseline/20261012-182813`
Figure 3	`locality/20261012-182813`
Figure 4	`alignment/20261012-182813`
Figure 5, Table 5	`stride/20261012-182813`
Figure 6	`dtype/20261012-182813`
Figure 7	`baseline/20261012-182813`, `negative/20261012-182813`
Table 4	`vectorization/20261012-182734`

Each raw directory holds `raw.csv` (one row per repetition, with full configuration metadata) and `env.txt` (a snapshot of `collect_env.sh`); the processed CSVs plotted above are produced from them by `tools/summarize` and copied into this report's `data/` by `scripts/sync_data.sh`. Every number quoted in the report traces to one of these files.